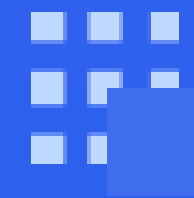


软件测试

变异测试优化技术



变异测试的挑战

- **作用：**执行变异评估测试集质量。
- **挑战：**大量变异体带来高昂成本。

```
def triangle(a, b, c):  
    if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):  
        print("无效输入值")  
    elif not (a + b > c and b + c > a and c + a > b):  
        print("无效三角形")  
    elif a == b == c:  
        print("等边三角形")  
    elif a == b or b == c or c == a:  
        print("等腰三角形")  
    else:  
        print("普通三角形")
```

对于第一个分支条件

not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100)

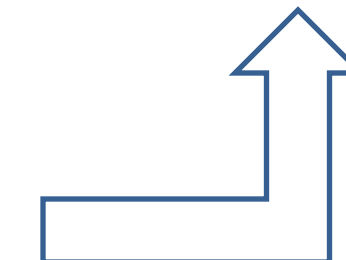
能够产生的变异体：

```
(1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100)  
not (1 < a <= 100 and 1 <= b <= 100 and 1 <= c <= 100)  
not (1 <= a <100 and 1 <= b <= 100 and 1 <= c <= 100)  
not (1 <= a <= 100 and 1 < b <= 100 and 1 <= c <= 100)  
not (1 <= a <= 100 and 1 <= b < 100 and 1 <= c <= 100)  
not (1 <= a <= 100 and 1 <= b <= 100 and 1 < c <= 100)  
not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c < 100)  
not (1 <= a <= 100 or 1 <= b <= 100 and 1 <= c <= 100)  
not (1 <= a <= 100 and 1 <= b <= 100 or 1 <= c <= 100)  
not (1 = a <= 100 and 1 <= b <= 100 and 1 <= c <= 100)  
.....
```

变异测试的优化

- **作用：**执行变异评估测试集质量。
 - **挑战：**大量变异体带来高昂成本。
- 一般来说，
- 构造变异体的成本较小
 - 执行变异体的成本较大

- 
- **变异体选择优化**
 - **变异体执行优化**

- 
- **变异算子选择**
 - **变异体随机选择**
 - **变异体聚类抽样**

变异体选择优化基本步骤

1. **准备测试集**：为待测程序准备测试集。
2. **创建变异体**：为待测程序创建变异体。
3. **运行测试**：运行测试，比较原程序和变异体的运行结果。
 - 如果结果不相同，变异体被该测试杀死。
 - 如果结果相同，变异体未被该测试杀死。
4. **重复步骤**：对测试集的其余测试重复上述步骤。

- **变异算子选择**
- **变异体随机选择**
- **变异体聚类抽样**

变异体选择的关键

策略

- 变异算子选择
- 变异体随机选择
- 变异体聚类抽样

思路

- 代表性变异体
- 冗余性变异体
- 真实性变异体

对于第一个分支条件

$\text{not } (1 \leq a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

能够产生的变异体:

$(1 \leq a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 < a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 \leq a < 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 \leq a \leq 100 \text{ and } 1 < b \leq 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 \leq a \leq 100 \text{ and } 1 \leq b < 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 \leq a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 < c \leq 100)$

$\text{not } (1 \leq a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c < 100)$

$\text{not } (1 \leq a \leq 100 \text{ or } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

$\text{not } (1 \leq a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ or } 1 \leq c \leq 100)$

$\text{not } (1 = a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

.....

1. $\text{not } (1 < a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

2. $\text{not } (1 = a \leq 100 \text{ and } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

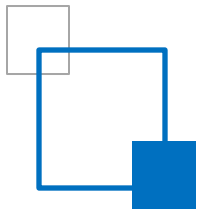
3. $\text{not } (1 \leq a \leq 100 \text{ or } 1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

4. $\text{not } (1 \leq b \leq 100 \text{ and } 1 \leq c \leq 100)$

思考:

如何定义代表性?

哪些算子更基本?



变异缺陷假设

变异测试优化技术

J.Offutt对早期Fortran语言的22个变异算子进行了系统性研究。
提出了五大类基本变异算子，被广泛应用于后续研究。

简写	描述	算子示例
ABS	绝对值插入	$\{(e, 0), (e, \text{abs}(e)), (e, -\text{abs}(e))\}$
AOR	算术操作符替换	$\{((aop\ b), a), ((aop\ b),\ b), (x, y) \mid x, y \in \{+, -, *, /, \% \} \wedge x \neq y\}$
LCR	逻辑连接符替换	$\{((aop\ b), a), ((aop\ b), b), ((aop\ b), \text{false}), ((aop\ b), \text{true}), (x, y) \mid x, y \in \{\&, , \wedge, \&\&, I\} \wedge x \neq y\}$
ROR	关系运算符替换	$\{((aop\ b), \text{false}), ((aop\ b), \text{true}), (x, y) \mid x, y \in \{>, >=, <, <=, ==, !=\} \wedge x \neq y\}$
UOI	一元操作符插入	$\{(\text{cond}, !\text{cond}), (v, -v), (v, \sim v), (v, --v), (v, v--), (v, ++v), (v, v++)\}$

变异缺陷假设 ■ 变异测试优化技术

变异测试优化目标

给定初始程序 P , 变异体集合 M ; 优化得到一个较小的变异体集合 M' , 使得对于不同的测试集 T , M' 对 T 的评估能力不比 M 差。

给定初始程序 P , 变异体集合 M ; 优化得到一个较小的变异体集合 M' , 使得对于不同的测试集 T , T 对 M' 的变异分数近似 T 对 M 的变异分数。

- 变异算子选择
- 变异体随机选择
- 变异体聚类抽样

给定初始程序 P , 两个变异体 m_1, m_2 ; 对于**任意**测试 t , t 对 m_1, m_2 的杀死情况相同, 则称 m_1, m_2 为等价; 对于**大多数**测试 t , t 对 m_1, m_2 的杀死情况相同, 则称 m_1, m_2 为相似。

相似则冗余

变异测试优化方法

- 变异算子选择：从经验上相似的变异算子中选择代表性的变异算子。
- 变异体随机选择：从经验上相似的变异体进行均匀随机抽样得到样本。
- 变异体聚类抽样：对变异体进行聚类分析然后进行随机抽样得到样本。

给定初始程序 P ，两个变异体 m_1, m_2 ；对于**任意**测试 t ， t 杀死 m_1 必然有 t 杀死 m_2 ，则认为 m_1 比 m_2 更难被杀死，则认为 m_1 强于 m_2 。

给定初始程序 P ，两个变异算子 o_1, o_2 ；对于**任意**测试 t ， t 杀死 $o_1(P)$ 必然有 t 杀死 $o_2(P)$ ，则认为 o_1 比 o_2 更难被杀死，则认为 o_1 强于 o_2 。

**选择强变异体
或者强变异算
子作为代表性**

回顾与讨论

